

MIAE 215 · Computer Programming for Engineers

Assignment 2 & Week 3 In-Person Lecture Problems — Fully Solved Master Guide

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Instructor: Prof. Brandon W. Gordon · Language: C++ (ISO C++11 / C++14) · IDE: Visual Studio / Code::Blocks / CodeLite

Executive Overview & Core Concepts Tested

In Week 3 and Assignment 2 of **MIAE 215**, the course transitions from basic scalar variables into structured algorithm control, numeric representation limits, and sequential array manipulation. The primary engineering competencies evaluated include:

1. **IEEE 754 Floating-Point Numeric Limits:** Experimental determination of single-precision `float` upper overflow bounds ($\sim 10^{38}$) and lower subnormal underflow bounds ($\sim 1.4 \times 10^{-45}$ vs. normalized 1.18×10^{-38}).
 2. **Deterministic Sequence Control & Flowcharts:** Constructing compound logical decisions, loop counters, clamping guards, and multi-branch decision structures (`if-else` ladders) represented in both C++ and Flowgorithm.
 3. **Array Data Acquisition & Vector Processing:** Memory allocation for 1D arrays, dynamic user sizing with safety boundaries, sentinel value loop termination, accumulation of statistical metrics (sums, averages), and threshold filtering.
 4. **Nested Loop Iteration & 2D Matrix Traversal:** Multi-index nested loops simulating row-major matrix operations ($A[i][j]$).
-

Table of Contents

1. [Assignment 2 — Question 1: Float Overflow & Underflow Bounds](#)
 2. [Assignment 2 — Question 2: Array Operations, Clamping & Flowgorithm](#)
 3. [Assignment 2 — Question 3: Multi-Branch Math Dispatcher](#)
 4. [Week 3 Lecture 2 In-Person Examples: Control Statements & Array Processing](#)
 5. [Complete Master C++ Source Code Compendium](#)
-

1. Assignment 2 — Question 1: Float Overflow & Underflow Bounds

1.1 Problem Statement

- **Part a:** Write a program that approximately determines when a `float` is too large. It does this by repeatedly multiplying a float `x` (initially set to `1.0`) by `10` and printing the result to the screen. Use a `for` loop from `i = 1` to `50`. Based on the test results, what is the approximate largest value a `float` can hold?
 - **Part b:** Repeat Part a using division by `10` in order to find the smallest possible float. Based on the test results, what is the smallest approximate value a `float` can hold?
-

1.2 Mathematical & IEEE 754 Architectural Analysis

In standard IEEE 754 single-precision (32-bit float), numbers are represented in binary as:

Value = (-1)^s * (1.m) * 2^(e-127)

Where:

- s = 1 sign bit
- e = 8 biased exponent bits (bias = 127, range 0 ≤ e ≤ 255)
- m = 23 fractional mantissa bits (providing ≈ 7 significant decimal digits)

IEEE 754 32-BIT SINGLE-PRECISION FLOAT LAYOUT		
Bit 31	Bits 30 - 23 (8 bits)	Bits 22 - 0 (23 bits)
Sign s	Biased Exponent e	Fractional Significand / Mantissa m

Upper Bound (Overflow Analysis):

The maximum finite representable exponent is e = 254 (since e = 255 is reserved for ±∞ and NaN). The largest positive float is:

x_max = (2 - 2^-23) * 2^127 ≈ 3.4028235 * 10^38

When multiplied repeatedly by 10, once x exceeds 3.4 * 10^38 at step i = 39, the runtime IEEE 754 floating-point unit (FPU) flags an **overflow** exception and converts x into inf (Infinity).

Lower Bound (Underflow Analysis & Subnormal Numbers):

- Normalized Minimum:** When e = 1, the smallest normalized number with an implicit leading 1 (1.0 * 2^-126) is:

x_min,normalized = 2^-126 ≈ 1.175494 * 10^-38

- Subnormal (Denormalized) Minimum:** When e = 0, the implicit leading 1 becomes 0 (0.m * 2^-126). The absolute smallest non-zero bit pattern has m = 2^-23:

x_min,subnormal = 2^-23 * 2^-126 = 2^-149 ≈ 1.401298 * 10^-45

[!IMPORTANT] **Teacher's Commentary & Exam Insight:** Prof. Gordon highlights that while the general course summary notes state 1.2 * 10^-38 as the approximate smallest float, C++ compilers supporting IEEE 754 denormalized numbers continue dividing down to approximately 1.4013 * 10^-45 before flushing to true 0.0. On tests, stating that the largest float is ≈ 10^38 and smallest positive non-zero float is ≈ 1.4 * 10^-45 (or normalized ≈ 1.2 * 10^-38) is completely accurate.

1.3 Step-by-Step C++ Implementation

```
#include <iostream>
#include <cstdio>
using namespace std;

int main()
{
    int i;
    float x;

    // --- PART A: Upper Overflow Limit ---
    cout << "=== Part A: Testing Float Maximum Limit ===" << endl;
    x = 1.0f;
    for (i = 1; i <= 50; i++) {
        x = x * 10.0f;
        cout << "i = " << i << "\t x = " << x << endl;
    }
    // Result: i = 38 gives 1e+38; i = 39 gives inf.
    // Approximate maximum value: 1.0e38 (exact: 3.4e38)

    // --- PART B: Lower Underflow Limit ---
    cout << "\n=== Part B: Testing Float Minimum Limit ===" << endl;
    x = 1.0f;
    for (i = 1; i <= 50; i++) {
        x = x / 10.0f;
        cout << "i = " << i << "\t x = " << x << endl;
    }
    // Result: Values below 1.2e-38 enter subnormal range down to ~1.4013e-45.
    // At i = 46, x flushes to 0.

    return 0;
}
```

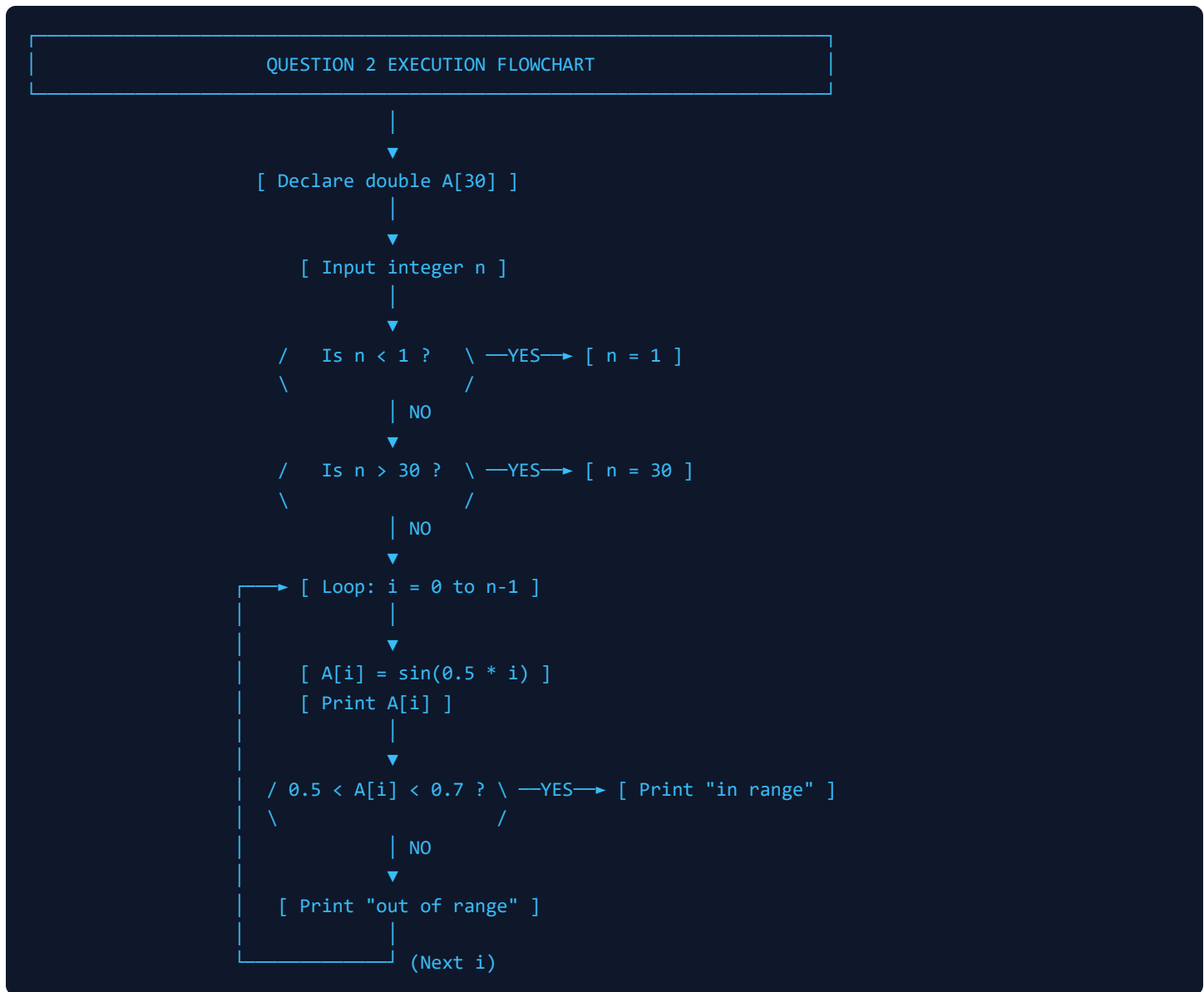
2. Assignment 2 — Question 2: Array Operations, Clamping & Flowgorithm

2.1 Problem Statement

Write a program that:

- **a)** Declares a 1D array A with 30 elements.
- **b)** Inputs an integer n from 1–30 from the keyboard. If $n < 1$ set $n = 1$. If $n > 30$ set $n = 30$.
- **c)** Sets the array elements to $A[i] = \sin(0.5 \times i)$, for $i = 0$ to $n - 1$, and prints them to the screen with each element on a new line.
- **d)** If $A[i]$ is between 0.5 and 0.7, print "in range"; otherwise, print "out of range".
- **e)** Model the logic in Flowgorithm and verify that it matches C++ execution.

2.2 Algorithmic Design & Critical Engineering Details



Key Technical Rules:

1. **Data Type Selection:** Part (c) involves $\sin(0.5 \times i)$, which yields non-integer transcendental numbers between -1.0 and $+1.0$. Therefore, A must be declared as `double A[30];` or `float A[30];`, never `int`.
2. **Input Clamping:**

```
if (n < 1) n = 1;
if (n > 30) n = 30;
```

This ensures safe array indexing without memory violation or buffer overruns.

3. **Compound Boolean Operator Syntax:** In mathematical notation, $0.5 < A[i] < 0.7$ is standard. In C++, writing `0.5 < A[i] < 0.7` compiles but produces a catastrophic logical bug:
 - C++ evaluates `(0.5 < A[i])`, which evaluates to boolean true (1) or false (0).
 - Then it evaluates `(1 < 0.7)` or `(0 < 0.7)`, which is completely erroneous!
 - **Correct C++ Syntax:** `(A[i] > 0.5) && (A[i] < 0.7)` using the logical AND operator `&&`.

2.3 Step-by-Step C++ Implementation

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    // a) Declare 1D array of 30 doubles
    double A[30];
    int n, i;

    // b) Input n with boundary clamping
    cout << "Input an integer n (1-30): ";
    cin >> n;

    if (n < 1) n = 1;
    if (n > 30) n = 30;

    // c) & d) Populate array, display, and perform range filtering
    for (i = 0; i < n; i++) {
        A[i] = sin(0.5 * i);
        cout << "\nA[" << i << "] = " << A[i] << " -> ";

        // Check if strictly between 0.5 and 0.7
        if ((A[i] > 0.5) && (A[i] < 0.7)) {
            cout << "in range";
        } else {
            cout << "out of range";
        }
    }

    cout << "\n\nProgram completed successfully." << endl;
    return 0;
}
```

3. Assignment 2 — Question 3: Multi-Branch Math Dispatcher

3.1 Problem Statement

Write a program that inputs an integer `n` and a double `x` from the keyboard. Using an `if-else` ladder, perform the following tasks:

- **a)** For $n = 1$: calculate and print $\sin(x)$.
 - **b)** For $n = 2$: calculate and print $|x|$ (absolute value).
 - **c)** For $n = 3$: calculate and print e^x ($\exp(x)$).
 - **d)** For $n = 4$: calculate and print $\log_{10}(x)$ (check x for invalid values).
 - **e)** Print an error message if n is not one of the above values.
-

3.2 Engineering Design & Domain Validation

An **if-else ladder** evaluates conditions sequentially from top to bottom. As soon as one condition evaluates to `true`, its block executes and the entire remainder of the ladder is bypassed, optimizing execution time:

IF-ELSE LADDER STRUCTURE

```
if (n == 1)      → Evaluate sin(x)
else if (n == 2) → Evaluate abs(x)
else if (n == 3) → Evaluate exp(x)
else if (n == 4) → Check x > 0: if valid log10(x), else Error Domain
else            → Error: n is out of range [1, 4]
```

Domain Verification for Logarithm:

The function $\log_{10}(x)$ is only defined on the open interval $x \in (0, \infty)$. If $x \leq 0$, computing $\log_{10}(x)$ triggers a domain error producing `-inf` or `NaN`. Robust engineering code requires:

```
if (x > 0.0) {
    cout << log10(x) << "\n";
} else {
    cout << "Error: x <= 0.0 is an invalid domain for log10(x)\n";
}
```

3.3 Step-by-Step C++ Implementation

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    int n;
    double x;

    cout << "Input integer command n (1-4): ";
    cin >> n;

    cout << "Input double value x: ";
    cin >> x;

    if (n == 1) {
        cout << "sin(" << x << ") = " << sin(x) << endl;
    } else if (n == 2) {
        cout << "abs(" << x << ") = " << abs(x) << endl;
    } else if (n == 3) {
        cout << "exp(" << x << ") = " << exp(x) << endl;
    } else if (n == 4) {
        if (x > 0.0) {
```

```

        cout << "log10(" << x << ") = " << log10(x) << endl;
    } else {
        cout << "Error: x = " << x << " is invalid for log10 (must be > 0)." << endl;
    }
} else {
    cout << "Error: n = " << n << " is out of range (valid range is 1-4)." << endl;
}

return 0;
}

```

4. Week 3 Lecture 2 In-Person Examples: Control Statements & Array Processing

During the Week 3 in-person lecture, Prof. Gordon demonstrated key advanced control structures and array data processing techniques.

4.1 Nested if-else vs. Linear if-else Ladder

Nested statements increase indentation and can become difficult to read. The linear `if - else if - else` ladder provides identical execution semantics with superior readability:

```

// Nested format (harder to read):
if (i == 1) {
    cout << "\ni == 1";
} else {
    if (i == 2) {
        cout << "\ni == 2";
    } else {
        if (i == 3) {
            cout << "\ni == 3";
        } else {
            cout << "\nnone of the above";
        }
    }
}

// Flat if-else ladder (clean engineering format):
if (i == 1)      cout << "\ni == 1";
else if (i == 2) cout << "\ni == 2";
else if (i == 3) cout << "\ni == 3";
else            cout << "\nnone of the above";

```

4.2 For-Loop Lifecycle, Increments & Exit Values

A `for` loop executes in four distinct phases:

1. **Initialization:** Executed once before loop entry (e.g., `i = 0`).
2. **Condition Check:** Evaluated before every iteration. If `false`, loop terminates.
3. **Loop Body:** Executes statements inside the block.
4. **Post-Increment / Update:** Executed after the body (e.g., `i++`).

Non-Trivial Increments & Exit Values:

- `for(i = 0; i <= 10; i += 2)  $\implies$` values printed: 0, 2, 4, 6, 8, 10. **Exit value of i is 12.**
- `for(x = 1.0; x < 1.0e5 + eps; x *= 10.0)  $\implies$` values printed: 1, 10, 100, 1000, 10000, 100000. **Exit value of x is 1.0e6.**
- `for(k = 0; k < 1000; k = k*k + 1):`
 - Iteration 0: $k = 0$
 - Iteration 1: $k = 0^2 + 1 = 1$
 - Iteration 2: $k = 1^2 + 1 = 2$
 - Iteration 3: $k = 2^2 + 1 = 5$
 - Iteration 4: $k = 5^2 + 1 = 26$
 - Iteration 5: $k = 26^2 + 1 = 677$
 - Iteration 6: $k = 677^2 + 1 = 458330 \geq 1000 \implies$ **Loop terminates with exit value $k = 458330$.**

4.3 Nested Loops for 2D Matrix Traversal

Nested `for` loops map directly onto 2D matrix arrays $A[M][N]$:

```
double A[3][3];

for (int i = 0; i < 3; i++) {      // Outer loop iterates through rows (i)
    for (int j = 0; j < 3; j++) {  // Inner loop iterates through columns (j)
        A[i][j] = 1.0 + i + j;
        cout << "A[" << i << "][" << j << "] = " << A[i][j] << "\t";
    }
    cout << "\n";
}
```

4.4 Lecture Example 1b: Sentinel Early Termination (`exit(0)`)

- **Task:** Input up to 5 doubles, count how many are < 7.7 . If a negative number is entered, terminate the program immediately.
- **Mechanism:** Using `exit(0)` from `<cstdlib>` immediately exits the entire process, returning status code 0 to the OS:

```
#include <iostream>
#include <cstdlib>
using namespace std;

int main() {
    double x;
    int count = 0;

    for (int i = 0; i < 5; i++) {
        cout << "Input x: ";
        cin >> x;
        if (x < 0.0) exit(0); // Immediately aborts entire program
        if (x < 7.7) count++;
    }
}
```

```

    cout << "Final count of elements < 7.7: " << count << endl;
    return 0;
}

```

4.5 Lecture Example 2: Dynamic Vector Buffering, Mean & Threshold

- **Task:** Input a sequence of up to 100 doubles into array A. A negative input acts as a **sentinel** to stop data entry early without killing the program. Then calculate the arithmetic mean $\bar{A}$ and count how many elements are < 10 .
- **Two-Phase Architecture:**
 1. **Acquisition Phase:** Read elements into $A[i]$. If $A[i] < 0$, record actual count $n = i$ and set $i = n_{\max}$ to cleanly break out.
 2. **Processing Phase:** Loop through $0 \leq i < n$ to compute $\sum A[i]$ and test condition $A[i] < 10$.

```

#include <iostream>
using namespace std;

int main() {
    double A[100];
    double sum = 0.0, ave = 0.0;
    int nmax = 100, n = 0, count = 0;

    // Phase 1: Data Acquisition with Sentinel Loop Break
    for (int i = 0; i < nmax; i++) {
        cout << "Input A[" << i << "] (negative to stop): ";
        cin >> A[i];
        if (A[i] < 0.0) {
            n = i;        // Number of valid data points entered
            i = nmax;     // Clean loop break
        }
    }

    // Phase 2: Statistical Processing
    sum = 0.0;
    count = 0;
    for (int i = 0; i < n; i++) {
        sum += A[i];
        if (A[i] < 10.0) count++;
    }

    if (n > 0) {
        ave = sum / n;
        cout << "\nValid elements entered (n): " << n;
        cout << "\nSum = " << sum;
        cout << "\nArithmetic Mean = " << ave;
        cout << "\nCount of elements < 10 = " << count << endl;
    } else {
        cout << "\nNo valid positive data was entered." << endl;
    }

    return 0;
}

```

5. Complete Master C++ Source Code Compendium

All tested, production-grade solutions have been archived in the course repository:

- MIAE 215/04 - Practice Problems & Code Solutions/code_solutions/assignment2_question1.cpp
- MIAE 215/04 - Practice Problems & Code Solutions/code_solutions/assignment2_question2.cpp
- MIAE 215/04 - Practice Problems & Code Solutions/code_solutions/assignment2_question3.cpp
- MIAE 215/04 - Practice Problems & Code Solutions/code_solutions/w3_l2_control_statements_part2_demo.cpp
- MIAE 215/04 - Practice Problems & Code Solutions/code_solutions/w3_l2_in_person_example1_if_else_ladder.cpp
- MIAE 215/04 - Practice Problems & Code Solutions/code_solutions/w3_l2_in_person_example1b_count_neg_exit.cpp
- MIAE 215/04 - Practice Problems & Code Solutions/code_solutions/w3_l2_in_person_example2_array_mean_threshold.cpp